Porramat Thaepngoen s4144787

Part 2 – 100%

Completing Part 2 of Assignment 2 marked an important milestone in my learning of object-oriented programming (OOP). At this stage, I successfully implemented both Level 1 and Level 2 requirements, which involved building core classes such as Guest, Product, ApartmentUnit, SupplementaryItem, Order, Records, Operations, and the newly added Bundle. Working through these levels deepened my understanding of how individual objects interact to form a functional system.

In Level 1, I focused on defining clear class hierarchies. I started with Product as the superclass, then extended it into subclasses ApartmentUnit and SupplementaryItem. This structure allowed me to practice inheritance and method overriding, and to see how shared behaviour (such as displaying product information) can be reused while still allowing subclasses to add their own attributes. For example, I kept the base attributes id, name, and price in Product, and added capacity only in ApartmentUnit. This decision followed good OOP design principles and reduced duplication.

When implementing Order and Records, I learned the value of object references over simple identifiers. Initially, I stored only product IDs, but I later switched to storing full product objects. This allowed me to directly access methods like get_price() or display_info(), which made my code more modular and intuitive. Designing the Records class as a central data manager helped me appreciate how encapsulation can isolate file-reading and searching logic from other parts of the program.

The second half of this part introduced more advanced concepts , such as custom exceptions, date validation, and the Bundle class, which pushed me to think more systematically about design and testing.

Implementing exception handling was a major learning experience. The specification required the program to detect errors such as non-alphabetic guest names, invalid quantities, and incorrect date sequences. I used try–except blocks together with loops that re-prompted the user until valid input was received. This taught me that good software design is not just about making a program run but ensuring it behaves reliably under all user conditions. I also wrote meaningful error messages so that users could clearly understand what went wrong and how to fix it.

For date handling, I used Python’s datetime module to automatically calculate the length of stay between check-in and check-out dates. This reduced manual errors and simplified the user input process. I also validated booking logic to prevent impossible scenarios such as check-in before the booking date or same-day check-in and check-out. While debugging

these conditions, I gained confidence in applying logical operators and condition chains effectively.

The Bundle class was the most challenging part of Level 2. I had to design a way to combine multiple existing products and apply for a discount of 20%. At first, I miscalculated by applying the discount incorrectly, but through testing I refined the logic to multiply the total price by 0.8. I also handled duplicate items within a bundle by grouping identical IDs and displaying their quantities neatly. This task showed me the importance of careful analysis and iterative testing, as even small logical mistakes could produce large discrepancies in output.

Creating a structured menu system within the Operations class helped me simulate a realistic user interface. Implementing clear display functions for guests and product lists trained me to think about usability and formatting.

Through this assignment, I also developed a stronger sense of professional coding habits. I began adding meaningful comments before each method to explain design decisions instead of merely describing syntax. I realized that documentation is part of communicating with future developers (or even my future self). Furthermore, following naming conventions and indentation consistently made my code more readable and aligned with the course's professional standards.

Part 2 strengthened my confidence in writing well-structured, maintainable Python programs. I learned how to combine inheritance, encapsulation, and exception handling to create a complete and logical system.

The biggest takeaway from this stage is that planning before coding saves significant time later. Drawing rough diagrams and testing each component before integration prevented many logical errors. Debugging now feels less like fixing mistakes and more like a method of learning how systems behave internally. Overall, finishing Part 2 gave me both technical proficiency and professional discipline that will support my growth in future programming courses and projects.

References

[1] W3Schools, "Python Classes and Objects," W3Schools.com, 2024. [Online]. Available: https://www.w3schools.com/python/python_classes.asp

[2] W3Schools, "Python Try Except," W3Schools.com, 2024. [Online]. Available: https://www.w3schools.com/python/python_try_except.asp